SEMANA 3 · AULA 1 · 03/11

SPEC, STEERING E SKILL

Da spec ao projeto, as regras da casa e a primeira skill de code review

Felipe Oliveira · IA Aplicada à Produtividade no Desenvolvimento

<LAB365> SENAI

A SEMANA EM UM SLIDE

A PERGUNTA

Como transformar o que eu sei em algo que o agente faz do mesmo jeito toda vez, em qualquer ferramenta?

A RESPOSTA

Três camadas de contexto: steering (regras da casa), skill (método) e spec (o que construir agora).

O DOMÍNIO

Code review com agentes, pedido da turma. O projeto é de vocês: nasce hoje de uma spec de 12 requisitos.

O JOGO

Disputa em toda aula: hoje a revisão cruzada; na quinta, o Torneio Red Team x Blue Team.

3

camadas de contexto

12

requisitos EARS na spec

3

ferramentas, mesmo conteúdo

1

torneio na quinta

TRÊS CAMADAS, TRÊS PERGUNTAS

STEERING

Como trabalhamos neste repositório?

escopo: o projeto inteiro **quando:** sempre, em todo turno **carrega:** regras, stack, convenções

SKILL

Como se faz este tipo de tarefa?

escopo: um tipo de tarefa, em qualquer repo **quando:** sob demanda, quando a descrição casa
carrega: método, checklist, scripts

SPEC

O que estamos construindo agora?

escopo: uma feature **quando:** uma vez, até entregar **carrega:** requisitos, design, tarefas

Leitura: do mais amplo e permanente para o mais estreito e efêmero. Os três convivem na mesma sessão.

AGENDA DA AULA

19h00	Org SENAI-IA: equipes, apelidos, logos e repos	GITHUB
19h10	As três camadas e a pergunta da semana	ABERTURA
19h15	Bloco 1: spec e steering	TEORIA
19h35	Atividade 1.1: da spec ao projeto	PRÁTICA
20h10	Intervalo	PAUSA
20h20	Bloco 2: skill, o método, a anatomia	TEORIA
20h45	Atividade 1.2: skill v0 + Disputa 1 cruzada	DISPUTA
21h20	Placar, o que fica, amanhã	FECHO

ENTREGÁVEIS DE HOJE

Projeto + steering + skill

- repo iapd-26t3-eNN-pagamentos
- pytest verde: 12 testes test_rNN
- steering da equipe
- skill code-review-<equipe>
- relatório da Disputa 1 cruzada

A ORG SENAI-IA NO GITHUB

TURMA E EQUIPES

Turma IAPD-26T3. Equipes IAPD-26T3-E01, E02...: o código não muda; o apelido vem depois do hífen.

REPOSITÓRIOS

Equipe: iapd-26t3-e03-<projeto>. Turma: iapd-26t3-materiais, uma pasta por aula.

TRABALHO INDIVIDUAL

Pasta com o seu usuário dentro do repo da equipe, branch própria, entrega por Pull Request.

PORTFÓLIO

No fim da turma os repos são arquivados e viram portfólio. Capriche no README.

AGORA, EM 10 MINUTOS

Cada equipe faz

- escolhe o apelido da equipe
- manda o logo (PNG quadrado)
- cria iapd-26t3-eNN-skills com README
- vincula o time como Admin
- dúvidas: @SENAI-IA/professores

BLOCO 1: SPEC E STEERING

O que construir agora, e as regras da casa que valem em toda tarefa

SPEC: O QUE CONSTRUIR AGORA

O QUE É

Requisitos, design e tarefas de UM projeto, escritos antes do código e lidos pelo agente ao executar.

O QUE VAI

Critérios de aceite verificáveis (EARS), decisões de arquitetura, lista de tarefas pequenas e ordenadas.

O QUE NÃO VAI

Regras do projeto (steering), procedimentos reutilizáveis (skill), código.

DURAÇÃO

Nasce com a feature e termina na entrega. Depois vira histórico, não contexto.

TESTE DE PERTENCIMENTO

Vai para a spec se...

- só vale para esta feature
- um humano precisa aprovar
- dá para verificar quando ficou pronto
- muda a cada feature

TRÊS ARQUIVOS, UM PROJETO

PASSO 1

requirements.md

O quê e por quê. 12 requisitos em EARS; cada um vira um teste test_rNN.

PASSO 2

design.md

Como. Stack, módulos, esquema do banco, sequências que merecem cuidado.

PASSO 3

tasks.md

Em que passos. 9 tasks pequenas; o agente executa uma a uma.

requirements.md (trecho)

```
R04  WHEN um reembolso for solicitado THE SYSTEM SHALL permitir apenas se a soma dos reembolsos <= valor pago;
     caso contrario SHALL rejeitar com erro 422 e nao alterar nada
R06  WHEN um webhook chegar com event_id ja processado THE SYSTEM SHALL responder sucesso sem processar de novo
```

STEERING: AS REGRAS DA CASA

O QUE É

Markdown no repo, lido pelo agente sem ninguém pedir. Fatos que valem para qualquer tarefa neste projeto.

O QUE VAI

Stack, comandos de build e teste, layout de pastas, convenções e gotchas curtos.

O QUE NÃO VAI

Procedimento longo (vira skill), regra que não pode falhar (vira hook), o que o modelo já sabe.

TAMANHO

Poucas dezenas de linhas. Entra em todo turno: cada linha custa em toda mensagem.

TESTE DE PERTENCIMENTO

Vai para o steering se...

- é verdade hoje e daqui a 3 meses
- vale para qualquer tarefa no repo
- o modelo erraria sem saber
- cabe em uma ou duas linhas

ONDE O STEERING MORA

FERRAMENTA	SEMPRE CARREGADO	CONDICIONAL POR ARQUIVO	OBSERVAÇÃO
Kiro	.kiro/steering/*.md (inclusion: always)	inclusion: fileMatch + padrão	também manual e auto
Claude Code	CLAUDE.md na raiz; .claude/rules/*.md	.claude/rules com paths:	CLAUDE.md em subpasta carrega ao entrar nela
VS Code + Copilot	.github/copilot-instructions.md	.github/instructions/*.instructions.md com applyTo:	lê CLAUDE.md e AGENTS.md também
Neutro: AGENTS.md	AGENTS.md na raiz	AGENTS.md aninhado por pasta	Copilot, Codex, Cursor, Gemini CLI

Agnóstico na prática: escreva um AGENTS.md curto na raiz e, se a sua ferramenta não lê, crie o arquivo dela com uma linha: "siga AGENTS.md".

O LOOP E A CHAMADA EVENTUAL

LOOP: próximo turno, tudo de novo, steering incluso

1. VOCÊ PEDE

"revisa esse PR"

2. JANELA DO TURNO, MONTADA DE NOVO A CADA VOLTA

system prompt | STEERING | histórico | arquivos | skill ativa

steering: toda volta, use ou não. **skill ativa:** só depois de chamada.

3. MODELO RESPONDE

e o loop recomeça

FORA DO LOOP: SKILLS INSTALADAS

Só name + description na janela, ~100 tokens cada

- `review-security`
- `pr-summary`
- `pytest-triage`
- `docs-writer`

CHAMADA EVENTUAL

Quando o pedido casa com a descrição, o corpo da skill (até ~5.000 tokens) entra na janela e fica até o fim da sessão. Se nunca casar, nunca paga.

Por isso: steering curto, porque paga em toda volta; skill pode ser longa, porque paga só quando chamada.

STEERING BOM, STEERING RUIM

AGENTS.md (pagamentos-api)

```
# pagamentos-api

- Python 3.10+, stdlib + pytest. Rode: python -m pytest -q
- Dinheiro é int em centavos. Nunca float.
- sqlite3 sem ORM; toda query com parametro ?
- Tokens: secrets.compare_digest, nunca ==
- Datas em UTC, timezone-aware
- PAN e token nunca vao para log
- Webhooks sao idempotentes (event_id unico)
- Antes de entregar: pytest verde
```

BOM

Fatos curtos

cada linha corrige um erro provável; barato e sempre presente

RUIM

Manual disfarçado

40 linhas de passos; ignorado após compactação e custa sempre

Sinal: se tem passos numerados, é procedimento. Procedimento é skill, não steering.

1.1: DA SPEC AO PROJETO

35 minutos: a equipe recebe a spec, escreve as regras da casa e constrói com o agente

DA SPEC AO PROJETO: COMO JOGAR

PASSO 1

Criar

Repo iapd-26t3-eNN-pagamentos na org, com README e o time como Admin.

PASSO 2

Steering

5 min: regras da casa no formato da sua ferramenta ou AGENTS.md. Antes do código.

PASSO 3

Spec

Dê requirements, design e tasks ao agente. Execute task por task; pytest a cada uma.

PASSO 4

Verde

12 testes test_r01 a test_r12 passando. Commit e push.

Plano B: se o agente travar ou o crédito acabar, a branch clean do repo iapd-26t3-pagamentos-api implementa a mesma spec. Use-a como base e siga.

BLOCO 2: SKILL

O método: procedimento empacotado, portável, carregado só quando a tarefa pede. E a anatomia por dentro

TRÊS JEITOS DE ENSINAR O AGENTE

NA CONVERSA
Prompt

vive	uma sessão
custo	repetir toda vez
falha	cada dev escreve diferente

NO PROJETO
Steering

vive	em todo turno
custo	tokens sempre
falha	vira procedimento gigante

NA PASTA
Skill

vive	quando a tarefa casa
custo	~100 tokens até ativar
ganho	método + scripts portáveis

Regra: fato curto e permanente vai para steering; procedimento vai para skill; ferramenta vai para MCP.

A DEFINIÇÃO OFICIAL

"Pastas organizadas de instruções, scripts e recursos que os agentes descobrem e carregam dinamicamente."

Zhang, Lazuka e Murag (Anthropic), Equipping agents for the real world with Agent Skills, 16/10/2025 (tradução nossa)

PASSO 1

Metadados

name + description de todas as skills, sempre no contexto. ~100 tokens cada.

PASSO 2

Instruções

O corpo do SKILL.md entra quando a tarefa casa com a descrição. < 5.000 tokens.

PASSO 3

Recursos

references/, assets/ lidos sob demanda; scripts/ executados sem entrar no contexto.

TAXONOMIA EM QUATRO EIXOS

MECANISMO	QUANDO CARREGA	QUEM DECIDE	O QUE CARREGA	CONTEXTO
steering sempre	todo turno	ninguém	regras	compartilhado
steering por arquivo	ao tocar arquivo que casa	padrão de arquivo	regras	compartilhado
skill	match de descrição ou /nome	modelo ou humano	método + scripts	compartilhado
plugin / power	palavra-chave da tarefa	modelo	skills + MCP tools	compartilhado
hook	evento: save, tool, stop	determinístico	comando ou prompt	fora do modelo
subagente	delegação	modelo ou nome	prompt + tools próprios	**ISOLADO**
MCP server	sempre registrado	modelo chama a tool	ferramentas	compartilhado

REGRA DE BOLSO: QUAL USAR



STEERING

Padrões do projeto

EXEMPLO

Centavos, sqlite com ?, UTC. Poucas linhas, sempre presentes.



SKILL

Procedimento reutilizável

EXEMPLO

Como revisar um PR; como triar falhas do pytest. Portável entre agentes.



PLUGIN

Ferramenta + método

EXEMPLO

Stripe: MCP tools + skills de checkout, ativadas por palavra-chave.



HOOK

Toda vez, sem falta

EXEMPLO

Ao salvar .py, roda pytest. Não depende do modelo lembrar.



SUBAGENTE

Contexto isolado

EXEMPLO

Três revisores em paralelo, cada um com sua skill e sua janela.



MCP

Ferramentas

EXEMPLO

Ler o Jira, consultar o banco. Dá o que fazer, não como.

UMA PASTA, VÁRIOS AGENTES

code-review/ (padrão agentskills.io)

```
code-review/
├── SKILL.md          # obrigatório
├── scripts/          # executa, não lê
├── references/       # lê sob demanda
├── assets/           # templates
└── evals/            # evals.json
```

Kiro	`.kiro/skills/`
Claude Code	`.claude/skills/`
VS Code + Copilot	`.github/skills/` (+ `.claude/skills/`)
Codex CLI	`.agents/skills/`

Spec aberta: name, description, license, compatibility, metadata, allowed-tools. Campos extras de um cliente são ignorados pelos outros, sem quebrar.

FRONTMATTER: CAMPOS E LIMITES

CAMPO	OBRIG.	LIMITE
name	sim	1 a 64, [a-z0-9-], sem -- e sem hífen nas pontas, igual à pasta
description	sim	1 a 1024 caracteres; o que faz e quando usar
license	não	nome da licença ou arquivo
compatibility	não	até 500; só se exigir ambiente
metadata	não	mapa chave: valor (strings)
allowed-tools	não	experimental; tools pré-aprovadas

SKILL.md

```
---
name: code-review
description: >
  Revisa diffs e PRs em Python
  procurando bugs de correção,
  segurança e desempenho. Use
  quando pedirem review, revisão,
  "olha esse PR" ou antes do merge.
  Não escreve testes.
metadata:
  author: turma-sd
  version: "0.1"
---
```

A DESCRIÇÃO CARREGA O GATILHO

ANTES

Não dispara ou dispara em tudo

texto	"Helps with code."
problema	não diz quando; sem palavras do usuário
efeito	o agente nunca escolhe, ou escolhe sempre

DEPOIS

Dispara no pedido certo

texto	"Revisa diffs e PRs em Python… Use quando pedirem review, revisão, 'olha esse PR' ou antes do merge. Não escreve testes."
por quê	o quê + quando + o que não faz
efeito	casa com o pedido, erra menos no near-miss

Teste mental: a descrição funcionaria se o usuário não dissesse a palavra 'review'? Se não, ela está estreita demais.

PROGRESSIVE DISCLOSURE EM NÚMEROS

~100

tokens por skill instalada (nível 1)

<5.000

tokens do corpo ao ativar (nível 2)

500

linhas: teto recomendado do SKILL.md

0

tokens de um script executado



COMO ESTRUTURAR

Diga quando ler cada arquivo

- "Leia references/cwe.md se encontrar entrada externa"
- não: "veja references/ para detalhes"
- referências a um nível de profundidade
- scripts para o que deve ser determinístico
- gotchas ficam no SKILL.md, não em reference

PADRÕES DE CORPO QUE FUNCIONAM

GOTCHAS

Fatos que o modelo erraria

EXEMPLO

"users usa soft delete: filtre deleted_at IS NULL". Corrigiu o agente? Vira gotcha.

TEMPLATE

Formato de saída concreto

EXEMPLO

A tabela de achados do README. O modelo casa padrão melhor que prosa.

CHECKLIST

Passos com dependência

EXEMPLO

[ ] coletar diff [ ] revisar [ ] validar [ ] só então entregar.

VALIDATION LOOP

Fazer, validar, corrigir

EXEMPLO

Rode validate_report.py; se falhar, corrija e rode de novo.

DEFAULT, NÃO MENU

Uma escolha e uma saída

EXEMPLO

"Use pdfplumber; se for escaneado, pdf2image + pytesseract."

PROCEDIMENTO

Como abordar, não a resposta

EXEMPLO

"1. leia o schema 2. junte por _id 3. filtre", não a query pronta.

CODE REVIEW: O QUE A SKILL CARREGA

SPEC E STEERING

A spec diz o que era para ser (R01 a R12); o steering, as regras da casa. A skill cita, não repete.

CATEGORIAS E SEVERIDADES

Seis categorias (segurança, correção, desempenho, dinheiro, concorrência, estilo) e quatro severidades.

FORMATO DO ACHADO

arquivo, linha, severidade, categoria, requisito violado (RNN), evidência, sugestão. Sem linha não conta.

LISTA DE CWES E O DIFF

references/cwe-top25.md e scripts/collect_diff.sh (diff + mensagem de commit).

YU ET AL. 2024/2026

O que a pesquisa diz

- 7 LLMs, 5 prompts, vs SAST
- LLMs superam ferramentas estáticas
- melhor prompt GPT-4: lista de CWEs
- melhor prompt R1: commit + passo a passo
- acham mais em arquivos com menos tokens

O ACHADO QUE VALE PONTO

reports/equipe-v0.md

```
| arquivo         | linha | severidade | categoria | requisito | evidencia                                  | sugestao        |
|-----------------|-------|------------|-----------|-----------|--------------------------------------------|-----------------|
| app/db.py       | 40    | critica    | seguranca | R09       | WHERE c.name = '{customer}' via f-string | parametro ?     |
| app/webhooks.py | 14    | alta       | seguranca | R08       | sig == expected                            | compare_digest  |
```



PASSO 1

Gerar

O agente escreve a tabela a partir da sua skill.



PASSO 2

Validar

python scripts/validate_report.py: formato, linha, severidade.



PASSO 3

Confirmar

A equipe dona confirma com um teste que falha, ou rejeita.

ATIVIDADE 1.2 E DISPUTA 1

Sua primeira skill de code review, contra o projeto da equipe vizinha

SKILL V0 E A REVISÃO CRUZADA



PASSO 1

Criar

Copie o template para a pasta de skills da sua ferramenta e preencha os TODOs.



PASSO 2

Validar

python validate.py na pasta code-review-<equipe>.



PASSO 3

Rodízio

Clone o repo da equipe vizinha. Sessão nova: "revisa este repositório contra a spec".



PASSO 4

Relatar

reports/<equipe>-v0.md no formato, com o requisito violado em cada achado.



PASSO 5

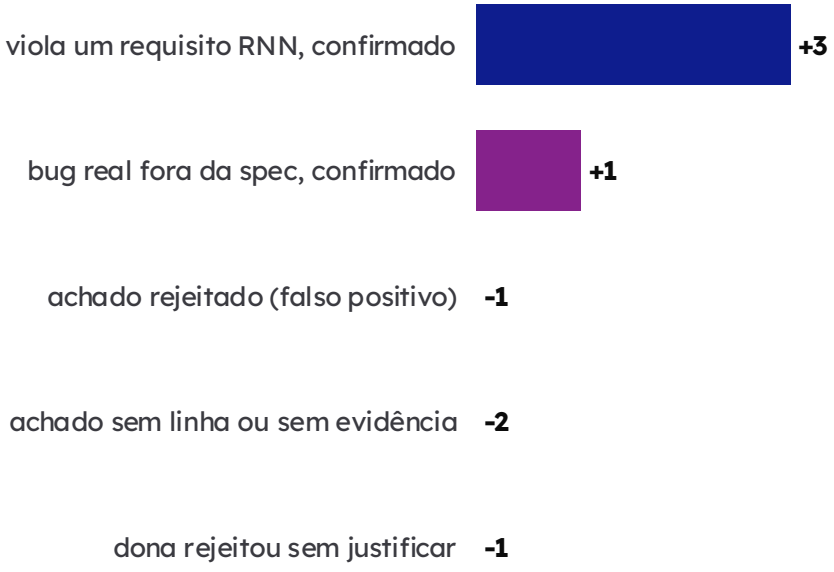
Confirmar

A dona do código confirma (teste que falha) ou rejeita cada achado.



Dica: a spec e o steering já dizem o que era para ser; a skill diz como revisar. Não repita uma na outra.

DISPUTA 1: REVISÃO CRUZADA



REGRAS

Revisão cruzada, rodada 1

- rodízio: 1 revisa 2, 2 revisa 3, ..., última revisa 1
- a spec é o gabarito: cite o RNN violado
- só vale achado no formato, com linha
- dona confirma com teste que falha, ou rejeita
- empate: o juiz decide pelo teste
- placar acumula até o Torneio

O QUE FICA E O QUE VEM

FICA

Spec = o que construir. Steering = regras da casa. Skill = método, sob demanda, portável.

NÃO APAGUE NADA

Amanhã a v0 vira uma suíte de skills finas de review e roda de novo sobre os mesmos projetos.

AMANHÃ (QUA 04/11)

Múltiplas skills: gatilho, granularidade, subagentes em paralelo, agente revisor.

QUINTA (05/11)

Spec, skill e steering do trabalho final; skills seguras; o Torneio Red x Blue.

12

requisitos na spec

?

achados confirmados hoje

72%

meta de amanhã (Rajan)

09/11

entrega da lista


<LAB365>
SENAI